

Master's Thesis — Working Draft

Memory Pruning and Forgetting Policies for AI Coding Agents

Impact on Performance Across Sequential Tasks

Leo Dinh

Master's Programme in Computer Science

DRAFT — compiled 2026-06-08.

Design chapters complete; empirical chapters await experiment execution.

Abstract

AI coding agents increasingly carry a persistent memory of past tasks, on the intuition that accumulated experience improves future performance. Yet unbounded accumulation is expensive and may actively hurt: longer context degrades reasoning, and stale or irrelevant memories act as retrieval noise. This thesis asks whether **proactive forgetting** — selectively pruning or consolidating stored experience — can match or beat full-memory accumulation on sequential software-engineering tasks while materially reducing cost, and **which structure** of forgetting matters most.

We evaluate six memory policies — No Memory, Full Memory, Random Prune, Recency Prune, Type-Aware Decay, and Cognitively-inspired (CLS) Consolidation — on all eight official SWE-Bench-CL sequences across three seeds (144 controlled runs), holding the retrieval mechanism **identical** across every condition so that the experimental contrast is cleanly attributable to the **storage** policy rather than the retrieval policy. Performance is measured with the continual-learning F1 (CL-F1) metric balancing plasticity and stability; cost is measured as a provider-independent compute proxy. The analysis is pre-registered: effect sizes (rank-biserial r_{rb}) with BCa bootstrap confidence intervals are primary evidence, Wilcoxon signed-rank tests on $N = 8$ sequence-level means supplement under Holm correction, and non-inferiority is tested with a TOST procedure against a pre-committed smallest-effect-size-of-interest of ± 0.03 CL-F1.

⚠ PENDING DATA — Empirical results are not yet available. This draft presents the complete experimental design, hypotheses, metrics, and analysis plan. The Results, Discussion, and Conclusion are scaffolded against the pre-registered interpretation rules and will be populated once the 144 runs execute.

Keywords: AI coding agents, agentic memory, catastrophic forgetting, continual learning, memory pruning, retrieval-augmented agents, SWE-Bench-CL, Pareto efficiency.

Tóm tắt (Vietnamese)

Các tác nhân lập trình AI ngày càng lưu giữ bộ nhớ bền vững về các tác vụ đã qua, với giả định rằng kinh nghiệm tích lũy sẽ cải thiện hiệu năng. Tuy nhiên, tích lũy không giới hạn vừa tốn kém vừa có thể gây hại: ngữ cảnh dài làm suy giảm khả năng suy luận, còn các ký ức cũ hoặc không liên quan trở thành nhiễu khi truy hồi. Luận văn này đặt câu hỏi liệu **sự quên chủ động** — cắt tỉa hoặc hợp nhất kinh nghiệm đã lưu một cách có chọn lọc — có thể **không thua kém** hoặc vượt trội so với việc tích lũy toàn bộ bộ nhớ trên chuỗi tác vụ kỹ thuật phần mềm, đồng thời giảm đáng kể chi phí, và **cấu trúc quên** nào là quan trọng nhất. Chúng tôi đánh giá sáu chính sách bộ nhớ trên toàn bộ tám chuỗi SWE-Bench-CL với ba seed (144 lần chạy có kiểm soát), giữ cơ chế truy hồi **giống hệt nhau** giữa các điều kiện. *[TODO: dịch hoàn chỉnh phần phương pháp & kết quả sau khi có dữ liệu].*

Contents

Abstract	2
1 Introduction	4
1.1 Motivation	4
1.2 Research question and sub-questions	4
1.3 Hypotheses	5
1.4 Contributions	5
1.5 Thesis structure	5
2 Background and Related Work	7
2.1 Agentic memory for LLM agents	7
2.2 Memory cost, context length, and retrieval noise	7
2.3 Continual learning and forgetting as adaptation	7
2.4 Positioning	7
3 Methodology	9
3.1 System architecture	9
3.2 Agent specification	9
3.3 Memory representation	9
3.4 The six memory policies	9
3.5 Retrieval (identical across all conditions)	11
3.6 Reflection and memory writing	11
4 Experimental Design	12
4.1 Benchmark and sequences	12
4.2 Experiment matrix	12
4.3 Metrics	12
4.4 Statistical analysis plan	12
4.5 Pareto analysis	13
4.6 Deviations from pre-registration	13
5 Results	14
5.1 Headline result table (scaffold)	14
5.2 Planned contrasts (scaffold)	14
5.3 Pareto frontier (scaffold)	14
5.4 Behavioral and memory-growth results (scaffold)	15
5.5 Failure case studies (scaffold)	15
6 Discussion	16
7 Threats to Validity	17
8 Conclusion	18
References	19

1 Introduction

1.1 Motivation

Modern AI coding agents are deployed across long streams of related tasks: fixing successive issues in the same repository, refactoring across a sprint, or maintaining a service over months. A natural design instinct is to let the agent **remember** — to persist a record of each task it solved and retrieve relevant prior experience when a new task arrives. The implicit promise is positive transfer: experience compounds, and the agent gets better over time.

This promise has a hidden cost structure. Persistent memory grows without bound unless something removes entries. As the store grows, three pressures appear. First, **operational cost** rises: every retrieved memory consumes input tokens, and larger stores require more retrieval and (for some designs) more maintenance computation. Second, **reasoning degrades**: a well-documented “lost-in-the-middle” effect [1] and the broader finding that longer context can hurt [2] mean that injecting more memory is not monotonically helpful. Third, **retrieval noise** accumulates: stale, repository-irrelevant, or superseded memories compete with useful ones for the limited top- k retrieval slots [3]. Recent work has even shown that naive add-all memory can be **worse** than no memory at all [4].

If accumulation is not free, then **forgetting** — deliberately discarding or compressing memories — becomes a first-class design decision rather than a fallback triggered by overflow. Cognitive science has long held that forgetting is adaptive rather than purely lossy: the power-law forgetting curve is a rational response to the statistics of information need [5], and active forgetting serves as a form of regularization [6]. This thesis imports that lens into agentic memory and asks the engineering question it implies: **given a fixed retrieval mechanism, which forgetting policy gives the best performance-per-cost on real sequential coding tasks?**

1.2 Research question and sub-questions

Central question. Under controlled retrieval, do proactive forgetting and consolidation policies achieve performance **non-inferior** to full-memory accumulation on sequential coding tasks while **materially reducing** operational cost — and which content-aware structure of forgetting matters most?

The central question decomposes into five sub-questions:

- SQ1** Which memory policies optimize the trade-off between sequential coding performance and operational cost?
- SQ2** Are gains explained mainly by reducing memory **volume** (Random), selectively retaining **useful** memory (Type-Aware), or **abstractive compression** (CLS)?
- SQ3** Which memory-item characteristics predict whether stored experience will help or harm future tasks?
- SQ4** How does agent performance change as persistent memory grows over time?
- SQ5** Does memory accumulation induce behavioral changes (tool-call frequency, syntax-error rates) that forgetting policies mitigate?

1.3 Hypotheses

The study pre-registers five falsifiable hypotheses (verbatim from the locked design, §1.3):

- H1** Selective pruning policies achieve at least one of: **(H1a)** non-inferior sequential performance vs. Full Memory, and/or **(H1b)** materially lower operational cost. H1a is tested for each pruning policy $P \in \{\text{Random, Recency, Type-Aware, CLS}\}$ as paired difference ($P - \text{Full}$) in CL-F1 across $N = 8$ sequences, evaluated by a **two one-sided tests (TOST)** procedure against a smallest-effect-size-of-interest $\text{SESOI} = \pm 0.03$ CL-F1 with a 95% BCa bootstrap CI (5000 iterations). H1b requires median paired $\Delta(\text{cost}) \leq -20\%$ with the 95% CI upper bound below zero. **Rejection rule (α -strict):** H1 is rejected for P iff H1a shows P degraded beyond the SESOI lower bound, regardless of H1b — performance non-inferiority is the load-bearing claim; cost reduction alone does not establish that forgetting “matches or beats” full memory.
- H2** Semantically informed pruning (Type-Aware Decay) outperforms random pruning — i.e. **what** is forgotten matters beyond simply reducing volume.
- H3** CLS Consolidation gives similar or marginally better raw performance than extractive pruning but at higher cost, **failing** the Pareto efficiency test against simpler policies.
- H4** Full-memory accumulation induces measurable **analysis paralysis** — increased tool calls and syntax errors — which forgetting policies mitigate.
- H5** Pruning can **harm** performance when it removes rare but critical repository-specific memories, identifying conditions under which forgetting is unsafe.

The SESOI of ± 0.03 CL-F1 corresponds to roughly one task in thirty changing outcome per sequence — the smallest practitioner-meaningful difference at SWE-Bench-CL’s typical sequence length. It is pre-registered before any data collection and locked.

1.4 Contributions

The contribution claim is **methodological**, not policy-set novelty (the six policies overlap in name with prior work; see Section 2). Concretely, this thesis delivers:

1. The first **Pareto map** of forgetting policies for coding agents on a standard continual-learning benchmark (SWE-Bench-CL), bracketed by both No-Memory and Full-Memory boundary baselines.
2. A clean **retrieval-as-control** design: retrieval scoring is held identical (pure cosine) across all six conditions ([invariant #5](#)), isolating the **storage** policy from the **retrieval** policy — a confound that prior work conflates.
3. A **pre-committed, effect-size-gated** interpretation protocol: seven triggered framings fixed before data collection (§19), so conclusions cannot be retrofitted to whichever policy happens to win.
4. A reusable, fully-logged experimental harness for memory-policy research, with per-task, per-memory-event, and trajectory logging sufficient for post-hoc helpful/harmful memory analysis.

1.5 Thesis structure

Section 2 situates the work against prior agentic-memory and continual-learning research. Section 3 specifies the system: agent, memory representation, the six policies, retrieval, and reflection. Section 4 fixes the experimental matrix, metrics, statistical plan, and the declared deviations from pre-registration. Section 5, Section 6, and Section 8 present results, interpre-

tation, and conclusions — **pending experiment execution**. Section 7 catalogues threats to validity.

2 Background and Related Work

2.1 Agentic memory for LLM agents

A growing body of work equips LLM agents with external memory. MemGPT [7] treats the context window as paged virtual memory; Generative Agents [8] store and retrieve a stream of observations with recency/importance/relevance scoring. MemoryBank [9] applies an Ebbinghaus forgetting curve to memory items — the closest cognitive-decay analog to our Type-Aware Decay policy, from which we differ by parameterizing decay **per memory type** rather than globally per item. A-MEM [10] organizes memory as a dynamically linked network formed at write time; we deliberately adopt a **static, content-based** taxonomy (Section 3.3) to keep the storage policy cleanly separable from retrieval ([invariant #5](#)). Reflexion [11] converts episodic outcomes into verbal feedback; our reflection step (Section 3.6) is methodologically inspired by it but emits **typed** structured records rather than free text.

2.2 Memory cost, context length, and retrieval noise

Several recent results motivate forgetting directly. Xiong et al. [4] show add-all memory can underperform no memory. Du et al. [2] document that longer context degrades performance, and Liu et al. [1] establish the “lost-in-the-middle” position effect that drives our injection-order decision ([best item last](#)). Cuconasu et al. [3] quantify how irrelevant retrieved passages act as noise. On the cost side, industry analyses of production memory systems [12] document runaway cost growth in accumulate-everything designs, and MemBench [13] benchmarks the cost dimension explicitly.

2.3 Continual learning and forgetting as adaptation

The continual-learning framing supplies our primary metric. We use SWE-Bench-CL [14], which structures SWE-Bench tasks into chronological per-repository sequences and defines plasticity, stability, and CL-F1. The cognitive-science foundation — that forgetting is a rational, adaptive process — comes from Anderson and Schooler’s power-law analysis of information need [5], Richards and Frankland’s “forgetting as regularization” [6], and the complementary-learning-systems (CLS) theory of McClelland et al. [15] that motivates our consolidation policy.

2.4 Positioning

The nearest work by name is Alqithami [16], which evaluates six forgetting policies — but on a custom privacy-aware generative-agent benchmark (FiFA), not coding, with no retrieval control and no estimation-based analysis. Table 1 summarizes the delta. Our novelty is **controlled measurement**, not policy invention: the combination of a standard CL coding benchmark, identical retrieval as confound control, No-Memory↔Full-Memory bracketing, and pre-committed effect-size-gated framings has not been assembled before.

Axis	Xiong 2025	Alqithami 2025	This work
Domain	LLM agents	Privacy agents	Coding (sequential)
Benchmark	mixed	FiFA (custom)	SWE-Bench-CL
# forgetting policies	1 vs 1	6	6
Retrieval held constant?	n/a	not stated	Yes — pure co-sine
No-/Full-Memory baselines	partial	absent	both
Primary metric	task success	composite	CL-F1
Statistical unit	per-task	per-run	per-sequence ($N = 8$)
Pre-registered framings	no	no	yes (§19)

Table 1: Positioning against the two closest prior works (abridged from v5 §1.4).

3 Methodology

3.1 System architecture

The system is a single LangGraph ReAct coding agent backed by a two-layer memory store (an active SQLite-backed record set plus a FAISS vector index over embedding payloads). For each task the pipeline is: **(1) retrieve** relevant memories under a token budget, **(2) build prompt context** with retrieved memories injected in relevance-sorted order (best item last), **(3) solve** the task with a bounded ReAct loop, **(4) evaluate** the produced patch with the standard SWE-Bench `eval_v3` harness, **(5) reflect** to produce a typed memory record, **(6) write** the record, and **(7) maintain** the store according to the active policy. Only steps (1), (6), and (7) — retrieve, write, maintain — differ between conditions; everything else is held identical.

3.2 Agent specification

The agent is a tool-using ReAct loop with a hard cap of `20 steps` per task (force-fail on exceed; invariant #3). It is given a fixed tool set (read/list files, search, run tests, apply edits, finish) and a prompt template that is byte-identical across all six conditions except for the injected memory block. The agent’s private chain-of-thought is **never** written to trajectory logs; only action summaries and observations are recorded.

3.3 Memory representation

Each memory is a structured `MemoryRecord` carrying the task it came from, the repository, the files touched, an outcome label, a **content type**, and an embedding payload. The content taxonomy is a locked five-type, **content-based** (not outcome-based) scheme (`invariant #7`):

Type	Tier	Meaning
architectural	Sacred	structural/design decisions about the repo
api_change	Critical	signature/interface changes
bug_fix	Important	a defect and its fix
test_update	Expendable	test-only changes
config	Expendable	configuration/build changes

Table 2: The locked 5-type content taxonomy (v5 §5.1). Type is orthogonal to outcome.

A type **classifier** (a separate temperature-0 structured-output LLM call) assigns the type; the “outcome” axis (resolved/failed) is kept strictly orthogonal and is never collapsed into the type.

The **embedding payload** is the single most safety-critical representation decision. It is restricted to `[Issue + Final Error + Final Diff]` and capped at `< 7500 tokens` (`invariant #4`), enforced by an assertion in the store. Raw trajectories are never embedded: the embedder would silently truncate them and produce garbage vectors.

3.4 The six memory policies

All six policies share an identical **retrieve** (see Section 3.5); they differ only in **write** and **maintain**. Table 3 summarizes them.

Policy	Write	Maintain (forgetting rule)
P0 No Memory	store nothing	—
P1 Full Memory	add all	never prune (boundary baseline)
P2 Random Prune	add all	archive a uniformly random victim over budget
P3 Recency Prune	add all	archive oldest first (keep most recent M)
P4 Type-Aware Decay	add all	archive lowest power-law score (see below)
P5 CLS Consolidation	add all	every $k = 5$ tasks, cluster + abstractively summarize old non-sacred memories

Table 3: The six conditions. Only write/maintain differ; retrieve is identical.

P1 Full Memory means “store everything; retrieve top- k under the same budget” — **not** “append everything into the prompt.” Appending all memories would confound the storage policy with context length and is reserved as an optional stress-test ablation.

P2 Random Prune isolates the pure **volume** effect: any improvement over Full Memory here comes from “less memory,” not “smarter memory.” It is the natural control for H2.

P4 Type-Aware Decay is the central semantic policy. When the active store exceeds the budget M , each record is scored with an Anderson-Schooler power-law form ([invariant #8](#) , locked formula):

$$\text{score} = \text{base}(\text{type}) \cdot \text{age}^{-d(\text{type})} \cdot (1 + \text{retrievals})^{0.5} \quad (1)$$

where **age** is tasks-since-creation, the frequency exponent 0.5 is sub-linear (doubling retrievals does not double value), and the per-type (base, d) weights are **theoretical and frozen at design time** (not calibrated from pilot data):

Type	base	decay d	tier
architectural	1.0	0.05	Sacred
api_change	0.8	0.15	Critical
bug_fix	0.6	0.25	Important
test_update	0.4	0.35	Expendable
config	0.3	0.40	Expendable

Table 4: Locked per-type decay parameters for Equation 1 (v5 §8 P4). Lowest score is archived first.

Crucially, the score does **not** include any success/failure-after-retrieval term: those are downstream **associated** labels ([invariant #10](#)), and using them in the policy would smuggle causal claims into the scoring.

P5 CLS Consolidation fires on a fixed schedule ([every \$k = 5\$ tasks](#) , invariant #9 — **not** trigger-on-overflow). It clusters old ($\text{age} \geq 10$), same-repo, non-architectural memories by file/similarity overlap and replaces each cluster of ≥ 3 records with one LLM-generated summary

capped at 350 tokens, archiving the originals with provenance links. If still over budget, it falls through to Type-Aware Decay. The consolidation LLM call is metered and charged to the cost axis — this is what H3 tests.

3.5 Retrieval (identical across all conditions)

Retrieval is the load-bearing experimental control. For every condition, the query is constructed identically and scored by **pure cosine similarity** with **no bonuses and no penalties** ([invariant #5](#)). The top- k memories (default $k = 5$) are selected under a token budget (default 2000 tokens) and injected **relevance-sorted with the best item last** ([invariant #6](#)), to counter the lost-in-the-middle effect [1]. Retrieval is restricted to the **same repository** in the main experiment ([invariant #16](#)). Because retrieval is byte-for-byte identical, any between-condition difference is attributable to **what was stored**, not **how it was retrieved**.

3.6 Reflection and memory writing

After each task the agent runs a single structured-output **reflection** call that emits one main typed record per task (deterministic outcome and file list), followed by the type-classifier call. The reflection is inspired by Reflexion [11] but produces a fixed schema so that downstream storage policies operate on uniform records.

4 Experimental Design

4.1 Benchmark and sequences

The primary (and only main-experiment) benchmark is **SWE-Bench-CL** [14]: eight MIT-licensed repository sequences of 15–80+ chronologically-ordered tasks each, evaluated by the standard SWE-Bench `eval_v3` Docker harness. All eight official sequences are used with **no re-ordering, no subsetting, and no self-generated sequences** ([invariant #1](#)), except where compute constraints force a documented exclusion (see Section 4.6).

4.2 Experiment matrix

The main experiment is locked at **6 policies $\times$ 8 sequences $\times$ 3 seeds = 144 controlled runs** ([invariants #1, #2](#)). Three seeds apply to **all six** conditions, not only the stochastic ones, so that within-condition variance is estimable everywhere. A 12-run cross-model exploratory probe is planned separately (Week 7) and is not part of the primary analysis.

4.3 Metrics

Correctness / continual learning. The primary metric is **CL-F1**, the harmonic mean of plasticity (online accuracy across all tasks) and an **anchor-probe** stability estimate:

$$\text{CL-F1} = \frac{2 \cdot \text{CL-Plasticity} \cdot \text{CL-Stability}}{\text{CL-Plasticity} + \text{CL-Stability}} \quad (2)$$

Because the full $a_{i,j}$ re-evaluation matrix is computationally infeasible at this scale ($O(10^4)$ extra agent runs), stability is estimated with a locked **anchor-probe schedule**: five anchor tasks per sequence are re-evaluated at four probe points against the memory snapshot taken at each point, bounding re-evaluations at $4 \times 5 = 20$ per (sequence, condition, seed). The anchor-probe stability **lower-bounds** the full-matrix stability; the full matrix is run only as a 4-cell sensitivity check if budget permits.

Efficiency. Total tokens, estimated cost, wall time, tool calls, test runs, files read/modified, and per-resolved-task normalizations.

Memory growth. Active/archived/consolidated counts and tokens over time, pruning ratio, consolidation ratio.

Behavioral (H4). Tool calls per task, syntax-error rate, files read per task, test runs per task — hypothesized to rise under Full Memory and stay bounded under forgetting.

Retrieval quality. Per-retrieved-memory `same_repo`, file overlap, type match, similarity, age, outcome, and rank; aggregated into Relevant, Retrieval-noise, and a stale-rate.

4.4 Statistical analysis plan

The philosophy is **estimation over testing** [17], [18]: with $N = 8$ independent sequences, NHST has very limited power, so effect sizes and confidence intervals are the primary evidence and p-values supplement.

- **Unit:** sequence-level means (averaged across 3 seeds), $N = 8$ paired observations per condition pair.

- **Effect size (primary):** rank-biserial r_{rb} ([invariant #12](#) — not Cohen’s d , not Cliff’s δ) and median paired difference with a 95% BCa bootstrap CI ([5000 iterations](#) , invariant #15).
- **Planned contrasts (5, pre-registered):** each pruning policy and No Memory vs. Full Memory, by paired Wilcoxon signed-rank ([invariant #11](#)) under Holm correction. The remaining 10 pairwise comparisons are exploratory with uncorrected p-values.
- **Non-inferiority (H1a):** TOST against SESOI = ± 0.03 CL-F1.
- **Task-level (exploratory):** a binomial-logit GLMM with crossed random effect on `task_id` ([invariant #14](#)).
- **Helpful/harmful prediction (SQ3):** PR-AUC with a VIF check ([invariant #13](#) — not accuracy, not ROC-AUC, because the positive class is 20%).

A worked reporting template (v5 §15.5) fixes the exact phrasing of each contrast so results are reported uniformly.

4.5 Pareto analysis

The headline output is a Pareto plot of **CL-F1 (y)** vs. **total system cost (x)**, where cost includes all operational LLM calls (agent + reflection + classifier + consolidation) but **excludes** the anchor-probe re-evaluation overhead (measurement cost not borne in deployment). Pareto-optimal (non-dominated) conditions become the practical recommendations. For CLS specifically we report cost-normalized CL-F1 per dollar; if CLS matches Type-Aware Decay on CL-F1 but costs multiples more, it fails the Pareto test (H3).

4.6 Deviations from pre-registration

The experiment executes on **Ollama Cloud** rather than the originally pre-registered GPT-5.4, an explicitly user-authorized, **declared** deviation (resource/access constraint) — not a silent design change. Five deviations are disclosed (Table 5). Each holds a factor **constant across all six conditions and three seeds**, so it is a fixed factor: between-policy comparisons (H1–H5) remain valid, but absolute resolution rates are **not** comparable to GPT-5.4 or public SWE-Bench leaderboards.

#	Change	Why it stays valid
D1	Model → qwen3-coder:480b (agent), gpt-oss:20b (summary/classifier)	held constant across conditions; absolute rates not leaderboard-comparable
D2	Embedder → local nomic-embed-text-v2-moe (768-d)	bounds the payload, not embedder identity; same embedder across all conditions
D3	Cost metric → token-count proxy (not USD)	Ollama is flat-rate; tokens are a provider-independent compute proxy
D4	Classifier structured output → JSON-mode + Pydantic validation	same 5-type, temp-0 task; classifier failure rate logged + handled identically
D5	Host → arm64 macOS, Docker linux/arm64; arm64-unbuildable tasks excluded by a deterministic build-probe	exclusion applied identically across conditions; per-sequence counts disclosed; escalate to x86_64 if >15% of any sequence unbuildable

Table 5: The five declared runtime deviations from the pre-registered design.

5 Results

⚠ PENDING DATA — No real experimental data exists yet. The integration spine (agent loop, eval harness, CL-F1 collection, run entry point) is partially built but has not executed a single valid run; all files currently in `results/raw/` are stub fixtures (identical hardcoded values, `run_id = "test_run_id"`). This section is scaffolded with the exact tables and figures the analysis plan will populate. **Do not cite any number below until the 144 runs complete.**

5.1 Headline result table (scaffold)

Policy	CL-F1	Plasticity	Stability	Total tokens
No Memory	—	—	—	—
Full Memory	—	—	—	—
Random Prune	—	—	—	—
Recency Prune	—	—	—	—
Type-Aware Decay	—	—	—	—
CLS Consolidation	—	—	—	—

Table 6: *[TODO: Mean $\pm$ SEM across 8 sequences $\times$ 3 seeds, once runs complete.]*

5.2 Planned contrasts (scaffold)

Contrast vs Full Memory	Δ CL-F1	r_{rb}	95% BCa CI	Holm-p
Random Prune	—	—	—	—
Recency Prune	—	—	—	—
Type-Aware Decay	—	—	—	—
CLS Consolidation	—	—	—	—
No Memory	—	—	—	—

Table 7: *[TODO: Five pre-registered contrasts; report TOST verdict per H1a alongside.]*

5.3 Pareto frontier (scaffold)

[TODO: Pareto scatter: CL-F1 (y) vs total token cost (x), 6 points + per-sequence SEM error bars.]

Figure 1: Pareto frontier of performance vs. cost — the headline figure.

5.4 Behavioral and memory-growth results (scaffold)

[TODO: Tool-calls-per-task and syntax-error-rate trajectories by policy (H4); active-memory-size-over-time curves; retrieval Relevant\ / noise\.]

5.5 Failure case studies (scaffold)

[TODO: Per-policy hand-coded cases: 5 memory-helped, 5 memory-irrelevant, 5 memory-harmful, 5 pruning-removed-useful, 5 consolidation-lost-detail (CLS), per the §18 template.]

6 Discussion

⚠ PENDING DATA — Interpretation depends on which outcome obtains. The design pre-registers seven **triggered framings** (v5 §19) so the narrative cannot be retrofitted to the winner. The scaffolding below records each framing; the realized framing will be selected by the locked combination rule once data exist.

The pre-committed framings, keyed to outcomes, are:

If Full Memory wins accumulation pays off at SWE-Bench-CL sequence lengths; forgetting’s value is cost, not performance — report H1b separately.

If Type-Aware Decay wins content-aware forgetting beats both volume reduction and accumulation (supports H2); examine which types drive it.

If Random Prune matches Type-Aware Decay the benefit is **volume**, not **semantics** — H2 not supported; a sobering, publishable null.

If Recency Prune wins a cheap temporal heuristic suffices; questions the value of semantic machinery.

If CLS loses on Pareto abstractive consolidation pays the LLM-on-write tax without a CL-F1 return at this sequence length (supports H3); gate on CLS firing rate ($<30\% \Rightarrow$ uninformative).

If No Memory is competitive memory adds little here — a strong negative result about the premise itself.

If the retrieval-overlap probe shows no contrast between storage policies the storage manipulation did not propagate to retrieval; all storage-policy claims are qualified accordingly.

[TODO: Populate with the realized framing; connect to SQ1–SQ5; integrate failure case studies; state which hypotheses are supported/rejected with effect sizes.]

7 Threats to Validity

The design anticipates twelve threats (v5 §20); the load-bearing ones:

Model stochasticity temperature-0 is approximately but not perfectly deterministic; mitigated by 3-seed pooling and by treating re-evaluation variance as within-sequence noise.

Retrieval confound addressed by construction — identical pure-cosine retrieval across conditions ([invariant #5](#)) — and **detected** by a retrieval-overlap (Jaccard) probe; if storage policies produce indistinguishable retrieved sets, all storage claims are qualified.

Full-Memory definition “store all, retrieve top- k ” rather than “append all,” to avoid confounding memory policy with context length.

Embedding truncation the <7500-token payload cap ([invariant #4](#)) prevents silent truncation; raw trajectories are never embedded.

Classifier accuracy the type classifier is audited and its failure rate logged and handled identically across conditions; a classifier-as-policy confound is explicitly tracked.

Type-Aware parameter calibration per-type decay d is **theoretical and frozen**, never tuned from pilot data, to preserve pre-registration.

Stability estimation anchor-probe stability lower-bounds the full matrix; a 4-cell full-matrix sensitivity check qualifies stability claims if estimates diverge by >0.05 .

Single benchmark + single base model external validity is bounded; the cross-model probe and any cross-repo ablation are exploratory only.

Causal interpretation memory-item helpful/harmful labels are **associated, not causal** ([invariant #10](#)); the helpful/harmful model is predictive, not explanatory.

Deviations D1–D5 absolute resolution rates are not comparable to GPT-5.4 / leaderboards; only **between-policy** contrasts are claimed (Section 4.6).

8 Conclusion

⚠ PENDING DATA — The conclusion follows from results. Scaffold: restate which forgetting structure (volume / semantic / abstractive) won the Pareto test, the answer to the central question (does proactive forgetting match-or-beat full memory at lower cost?), the supported/rejected hypotheses with effect sizes, and the bounded scope of the claim (single benchmark, single base model, declared deviations). End with the practical recommendation: the Pareto-optimal policy(ies) practitioners should adopt.

[TODO: Write once H1–H5 verdicts exist.]

References

- [1] N. F. Liu and et al., “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, 2024.
- [2] e. a. Du, “Context length hurts: performance degradation with longer context,” 2025.
- [3] F. Cuconasu and et al., “The Power of Noise: Redefining Retrieval for RAG Systems,” 2024.
- [4] e. a. Xiong, “Add-all memory can be worse than no memory for LLM agents,” 2025.
- [5] J. R. Anderson and L. J. Schooler, “Reflections of the environment in memory,” *Psychological Science*, 1991.
- [6] B. A. Richards and P. W. Frankland, “The persistence and transience of memory,” *Neuron*, 2017.
- [7] C. Packer and et al., “MemGPT: Towards LLMs as Operating Systems,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [8] J. S. Park and et al., “Generative Agents: Interactive Simulacra of Human Behavior,” in *UIST*, 2023.
- [9] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, “MemoryBank: Enhancing Large Language Models with Long-Term Memory,” in *AAAI Conference on Artificial Intelligence*, 2024.
- [10] W. Xu *et al.*, “A-MEM: Agentic Memory for LLM Agents,” 2025.
- [11] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [12] Fastpaca, “Memory cost explosion in production agent systems.”
- [13] MemBench, “MemBench: Benchmarking the cost of agent memory,” 2025.
- [14] T. Joshi and et al., “SWE-Bench-CL: Continual Learning for Software Engineering Agents,” 2025.
- [15] J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly, “Why there are complementary learning systems in the hippocampus and neocortex,” *Psychological Review*, 1995.
- [16] S. Alqithami, “Forgetful but Faithful: A Cognitive Memory Architecture and Benchmark for Privacy-Aware Generative Agents,” 2025.
- [17] G. Cumming, “The New Statistics: Why and How,” *Psychological Science*, 2014.
- [18] R. L. Wasserstein, A. L. Schirm, and N. A. Lazar, “Moving to a World Beyond $p < 0.05$,” *The American Statistician*, 2019.